Techinical Appendix

Hintz A.

2026-08-19

Contents

1 Introduction **1**

2 Required R packages **2**

3 Data **2**

4 Modeling profit **3**
4.1 Canonical analysis 5
4.2 Loss function 7

5 Stan implementation **7**

6 Data fitted **12**

7 MCMC convergence diagnostics **13**

8 Posterior predictive checking **17**

References **18**

1 Introduction

This appendix provides the documentation and computer code used to implement the Bayesian decision-theoretic framework presented in the paper (…). The objective of the analysis is to evaluate nitrogen (N) and irrigation management strategies under uncertainty.

Rather than selecting management decisions solely according to predicted profit, the proposed framework combines posterior predictive distributions with an asymmetric loss function to quantify the expected consequences of alternative management decisions. This approach allows us to identify optimal management regions rather than a single optimal combination of inputs. The resulting recommendations account for expected profit, resource-use efficiency, uncertainty, and decision-maker risk preferences.

Table 1: Variables

variable	n_missing	mean	sd	min	median	max
year	0	4.02	1.61	1.00	4.00	6.00
N	0	178.60	52.30	0.00	185.00	370.00
water	0	20.06	4.44	9.45	20.41	31.10
profit	0	286.26	154.95	-245.93	310.83	592.34

The framework is illustrated using multi-year data from the TAPS network. The data used in this appendix are simulated data constructed to reproduce the structure required by the proposed methodology and to facilitate demonstration and visualization of the analysis.

The statistical model was fitted using Bayesian methods, with posterior distributions obtained through Markov chain Monte Carlo (MCMC) algorithms (Brooks 1998). Model adequacy was evaluated using convergence diagnostics, posterior predictive checks, and residual analysis. Convergence was assessed using graphical diagnostics and summary statistics, including the potential scale reduction factor $\widehat{R}$ (Gelman et al. 2013).

The Bayesian model was implemented in `Stan` (Stan Development Team 2026), while data processing, model fitting, posterior analysis, and graphical development were performed in `R` (R Core Team 2026).

This appendix is organized to mirror the analysis workflow: …

2 Required R packages

The following packages are required to reproduce the analysis:

```
library(tidyverse)
library(skimr)
library(cmdstanr)
library(bayesplot)
library(ggridges)
library(kableExtra)
library(posterior)
library(knitr)
library(readxl)
```

3 Data

The data we used to illustrate our models were generated by the Nebraska TAPS field experiments. The field experiments were conducted during the 2017 through 2022 growing seasons at experimental fields located in North Platte, Nebraska, USA. For a new application, this is the point at which the user substitutes their own field-trial data, provided it contains the same variables described Table 1.

The dataset contains 492 observations and four variables. The variable *year* identifies the experimental year, N represents the amount of nitrogen applied during the growing

season, *water* represents the total amount of water available from precipitation plus irrigation, and *profit* represents the resulting crop profit.

The response surface model contains linear, quadratic, and interaction terms involving nitrogen and water. Because these variables are measured on different scales, they are standardized with mean 0 and variance 1 to facilitates numerical stability during estimation. For each management variable, the standardized value is calculated as

$$x_k = \frac{X_k - \bar{X}k}{sX_k}, \qquad k = 1, 2,$$

where X_1 and X_2 denote the observed nitrogen and water levels, respectively, $\bar{X}k$ is the sample mean, and sX_k is the sample standard deviation.

The transformation is implemented in R as:

```
df <- df |>
  mutate(
    X1 = as.numeric(scale(N)),
    X2 = as.numeric(scale(water)))
```

A regular grid covering the observed range of standardized nitrogen and water levels is constructed to evaluate the fitted response surface over the management space.

```
grid_N <- seq(min(df$X1), 4, length.out = 100)
grid_W <- seq(min(df$X2), 4, length.out = 100)
grid <- expand.grid(N = grid_N, W = grid_W)
```

The resulting grid contains 100 values for nitrogen and 100 values for water, producing 10,000 combinations of management inputs. These combinations are subsequently used to calculate posterior predictions, expected profit, and derivatives quantities.

4 Modeling profit

To model profit as a function of N and irrigation management, we adopted a Bayesian response surface methodology (RSM) framework. The model also includes a year-specific random effect to account for variation among experimental years. Since profit is a continuous variable and may exhibit asymmetric behavior, we assume that it may follow a skew-normal distribution (Azzalini 2013):

$$[y_{ij} \mid \mu_{ij}, \sigma, \alpha] \sim \text{SkewNormal}(\mu_{ij}, \sigma, \alpha),$$

where y_{ij} denotes the observed profit for observation i in year j, μ_{ij} is the location parameter, σ is the scale parameter, and α controls the skewness of the distribution. The likelihood is implemented in Stan as:

```
  y ~ skew_normal(mu, sigma, alpha)
```

The scale parameter is constrained to be positive in the Stan parameter block and is assigned the prior

$$\sigma \sim \text{Gamma}(1100, 10).$$

which is implemented as:

```
real<lower=1e-6> sigma;
sigma ~ gamma(1100, 10);
```

The skewness parameter is assigned the prior

$$\alpha \sim \mathcal{N}(-2, 10)$$

which is implemented as:

```
real alpha;
alpha ~ normal(-2, 10);
```

The location parameter of the skew-normal distribution is modeled using a second-order response surface

$$\mu_{ij} = \beta_0 + v_j + \beta_1 x_{1ij} + \beta_2 x_{2ij} + \beta_3 x_{1ij}^2 + \beta_4 x_{2ij}^2 + \beta_5 x_{1ij} x_{2ij}, \tag{1}$$

where x_{1ij} and x_{2ij} represent the standardized nitrogen and water levels, respectively. The corresponding Stan implementation is:

```
 for (i in 1:N) {
    mu[i] = b0 + v[year[i]]
            + b1 * x1[i] + b2 * x2[i]
            + b3 * square(x1[i]) + b4 * square(x2[i])
            + b5 * (x1[i] * x2[i])
}
```

The correspondence between the mathematical notation and the Stan implementation is

$$(\beta_0, \beta_1, \beta_2, \beta_3, \beta_4, \beta_5) \longleftrightarrow (\texttt{b0}, \texttt{b1}, \texttt{b2}, \texttt{b3}, \texttt{b4}, \texttt{b5}).$$

Thus, b0 represents the intercept, b1 and b2 represent the linear effects of N and water, b3 and b4 represent the quadratic effects, and b5 represents the N and water interaction.

To account for differences among experimental years, the model includes a year-specific random intercept:

$$v_j =\sim \mathcal{N}(0, \sigma_v),$$

where

$$\sigma_v \sim \text{Half-Normal}(0, 10).$$

Therefore the intercept is not centered in β_0 but vary according the year. In the Stan model, the standard deviation is constrained to be positive and the standardized year effects are sampled independently:

```
real<lower=0> sigma_v;

vector[K] v_j;
```

The appropriate year effect (v_j) is included in the predictor through `v[year[i]]`. The prior for the year-level standard deviation is implemented as

```
sigma_v ~ normal(0, 10);
v_j ~ normal(0, sigma_v);
```

The prior distributions for the coefficients unknown were calibrated using information obtained from previous field data with varying nitrogen and water levels. Following the general approach of Little (2006), a classical frequentist model was first fitted to comparable data, and the resulting estimates were used to inform the prior distributions for the Bayesian model. The priors used in the Stan implementation are:

$$\beta_0 \sim \mathcal{N}(236, 15), \qquad \beta_1 \sim \mathcal{N}(-6, 11), \qquad \beta_2 \sim \mathcal{N}(60, 13),$$
$$\beta_3 \sim \mathcal{N}(-10, 13), \qquad \beta_4 \sim \mathcal{N}(-17, 9), \qquad \beta_5 \sim \mathcal{N}(-19, 15).$$

and in Stan:

```
real b0;
real b1;
real b2;
real b3;
real b4;
real b5;

b0 ~ normal(236, 15);
b1 ~ normal(-6, 11);
b2 ~ normal(60, 13);
b3 ~ normal(-10, 13);
b4 ~ normal(-17, 9);
b5 ~ normal(-19, 15);
```

4.1 Canonical analysis

We produced derived quantities (i.e., functions of the parameters in Equation~1), to characterize the N-W input combinations associated with maximum profit. This constitutes a Bayesian approach to canonical analysis, in which the stationary point is described by a posterior distribution rather than a single point estimate. Consider that (1) can be rewritten in matrix form as

$$\mu = \beta_0 + v_j + x'b + x'Bx, \tag{2}$$

where $b = [\beta_1, \beta_2]$ contains the linear coefficients and is implemented as `b_vec`:

```
vector[2] b_vec;
b_vec[1] = b1;
b_vec[2] = b2;
```

B is the 2×2 symmetric matrix of second-order coefficients given by

$$B = \begin{bmatrix} \beta_3 & \frac{1}{2}\beta_5 \\ \frac{1}{2}\beta_5 & \beta_4 \end{bmatrix}$$

and is implemented as B_mat:

```
matrix[2, 2] B_mat;
B_mat[1, 1] = b3;
B_mat[1, 2] = b5 .* 0.5;
B_mat[2, 1] = b5 .* 0.5;
B_mat[2, 2] = b4;
```

The stationary point of the response surface is obtained by differentiating Equation~2 with respect to x and setting the resulting gradient equal to zero. Solving the resulting system yields the stationary point

$$x_s = -\frac{1}{2}B^{-1}b, \tag{3}$$

and coded as:

```
vector[2] x_s;
x_s = -0.5 * (B_mat \ b_vec);
```

The eigenvalues and eigenvectors of B characterize the nature of the stationary point and define the principal directions of the response surface. Their relationship is given by

$$P'BP = \Lambda,$$

where Λ is the diagonal matrix containing the eigenvalues of B. Stan provides the function `eigenvalues_sym()` to calculate the eigenvalues of a symmetric matrix:

```
vector[2] eigen_val;
eigen_val = eigenvalues_sym(B_mat);
```

The columns of P are the corresponding orthonormal eigenvectors. In Stan, they are calculated in a matrix form using `eigenvectors_sym()`:

```
matrix[2, 2] eigen_vec;
eigen_vec = eigenvectors_sym(B_mat);
```

4.2 Loss function

The squared loss function is among the most commonly used loss functions in the literature. Under the squared loss (blue curve in Figure 1), deviations above and below the optimal profit are penalized equally. In this study we propose using the Linex loss function, following (Cressie 2023, williams2016combining), which penalizes outcomes farther from the optimum more heavily than the quadratic loss does (black curve, Figure 1). The loss captures the difference between $y_{pred\ i}$, the profit associated with each combined N-W action, and θ, the optimal profit obtained through the canonical analysis. In this way, the Linex loss can be written as

$$L(y_{pred\ i}, \theta) = \exp\big[c \cdot (y_{pred\ i} - \theta)\big] - c \cdot (y_{pred\ i} - \theta) - 1, \tag{4}$$

where c is a scaling parameter controlling the overall magnitude and direction of the asymmetry. In additional, the function penalizes actions that fail to reach maximum profit more heavily (i.e., when $y_{pred\ i} < \theta$) than action that exceed the optimum ($y_{pred\ i} > \theta$), reflecting the asymmetric economic cost.

In Stan the square loss and the linex loss can be written as function block

```
// LINEX Loss
real LINEX(real y, real theta, real c) {
  return exp(c * (y - theta)) - c * (y - theta) - 1;
}

//Square Loss
real SL(real y, real theta, real c) {
   return (c * (y - theta)) ^ 2;
}
```

where y is y_{predi} and theta is θ.

5 Stan implementation

Stan is an open-source probabilistic programming language for Bayesian statistical modeling. It allows users to specify statistical models, estimate posterior distributions using Markov chain Monte Carlo (MCMC) methods, and quantify uncertainty in model parameters and predictions. Stan can be integrated with R through interfaces such as `cmdstanr`, facilitating model fitting and posterior analysis. The model is divided into six blocks: `functions`, `data`, `parameters`, `transformed parameters`, `model`, and `generated quantities`.

- function{} defines the custom functions used in the model. In this case contains the loss function used in the decision-theoretic analysis.
- data{} specifies the observed response variable and predictors used to fit the model. It also include the N and W grid used for posterior prediction.

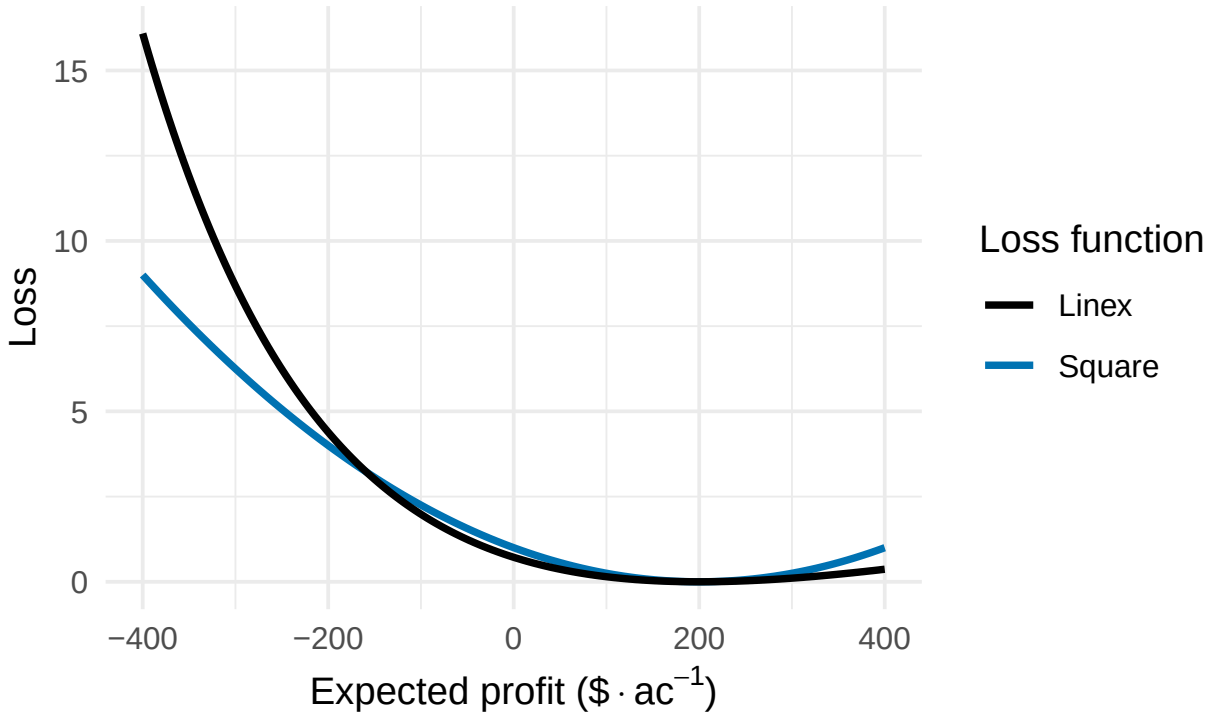


Figure 1: Comparison between the Linex loss function (black line) and the squared loss function (blue line) as a function of expected profit given the optimum profit *heta* is 200.

- parameters{} declares the unknown model parameters to be estimated, including their types and constraints.
- transformed parameters{} defines derived variables used in the model. Although these calculations could be included directly in the model block, placing them here makes the code more organized and easier to understand;
- model{} specifies the prior distribution and likelihood used to fit the bayesian model and obtain the posterior distribution of the parameters
- generated quantities{} calculates quantities of interest from the posterior draws after model fitting. In this study, these include the stationary point, eigenvalues and eigenvectors, the response surface coefficients (**b** and **B**), predicted profit over the N–W grid, LINEX loss values, and the predicted response at the maximum point.

The stan complete can be coded as

```
mod <- "

functions {
  // LINEX Loss
  real LINEX(real y, real theta, real c) {
```

```
    return exp(c * (y - theta)) - c * (y - theta) - 1;
  }
}

data {
  int<lower=0> N;
  int<lower=0> N_test;
  int<lower=1> K;          // number of mixture components (Year)
  array[N] int<lower=1, upper=K> year;   // year id per observation

  //y-data
  vector[N] y; //profit

  //x-data
  vector[N] x1; //Nitrogen
  vector[N] x2; //Irrigation

  real c;

  vector[N_test] act_x1; //N test
  vector[N_test] act_x2; //I test
}


parameters {
  //unknown parameters
  real b0;
  real b1;
  real b2;
  real b3;
  real b4;
  real b5;

  real<lower=1e-6> sigma;
  real alpha;

  real<lower=0> sigma_v;
  vector[K] Z_j;

}

transformed parameters {
  vector[N] mu;
  vector[K] v = sigma_v * Z_j;
```

```
  for (i in 1:N) {
    mu[i] = b0 + v[year[i]]
            + b1 * x1[i] + b2 * x2[i]
            + b3 * square(x1[i]) + b4 * square(x2[i])
            + b5 * (x1[i] * x2[i]);
  }
}

model {
  target += normal_lpdf(b0 | 236, 15);
  target += normal_lpdf(b1 | -6, 11);
  target += normal_lpdf(b2 | 60, 13);
  target += normal_lpdf(b3 | -10, 13);
  target += normal_lpdf(b4 | -17, 9);
  target += normal_lpdf(b5 | -19, 15);


  target += gamma_lpdf(sigma | 1100, 10);
  target += normal_lpdf(alpha | -2, 10);

  // Likelihood
  target += normal_lpdf(sigma_v | 0, 10);
  target += normal_lpdf(Z_j | 0, 1);
  target += skew_normal_lpdf(y | mu, sigma, alpha);
}


generated quantities {
  vector[N] y_rep;
  vector[2] b_vec;
  vector[2] x_s;
  vector[2] eigen_val;
  vector[N_test] grid_pred;
  vector[N_test] loss_linex;

  real profit_opt;

  matrix[2, 2] B_mat;
  matrix[2, 2] eigen_vec;

  b_vec[1] = b1;
  b_vec[2] = b2;

  B_mat[1, 1] = b3;
```

```
  B_mat[1, 2] = b5 * 0.5;
  B_mat[2, 1] = b5 * 0.5;
  B_mat[2, 2] = b4;

  x_s = -0.5 * (B_mat \\ b_vec);

  //Canonical analysis
  eigen_val = eigenvalues_sym(B_mat);
  eigen_vec = eigenvectors_sym(B_mat);


  // Posterior predictive check
  for (n in 1:N) {
    y_rep[n] = skew_normal_rng(mu[n], sigma, alpha);
  }

  // Profit optim
  profit_opt = b0
              + x_s[1] * b1
              + x_s[2] * b2
              + square(x_s[1]) * b3
              + square(x_s[2]) * b4
              + x_s[1] * x_s[2] * b5;


  // Profit grid predict
  for (i in 1:N_test) {

    grid_pred[i] = b0
                   + act_x1[i] * b1
                   + act_x2[i] * b2
                   + square(act_x1[i]) * b3
                   + square(act_x2[i]) * b4
                   + act_x1[i] * act_x2[i] * b5;

  // loss
  loss_linex[i] = LINEX(grid_pred[i], profit_opt, c);
  }

}

"
```

The Stan code is written to a `.stan` file using `write_stan_file()` and compiled with `cmdstan_model()`. After compilation, posterior sampling is performed using the MCMC

sampler implemented in Stan. Multiple chains are used to assess convergence and obtain posterior draws for all model parameters and generated quantities.

The posterior draws are then used for model inference and decision analysis. In addition to the regression coefficients and variance parameters, the Stan model directly generates posterior samples of the stationary point, eigenvalues and eigenvectors from the canonical analysis, predicted profit across the N–W grid, and the corresponding LINEX loss. Posterior predictive samples (`y_rep`) are also generated to evaluate the ability of the fitted model to reproduce the observed profit distribution.

```
stan_file <- write_stan_file(mod)
model <- cmdstan_model(stan_file)
```

6 Data fitted

The data are organized as an R list to match the variables declared in the Stan data{} block. The list contains the number of observations (N), the number of N–W combinations used for prediction (N_test), the number of years (K), the year associated with each observation, the observed profit (y), and the N and W predictors (x1 and x2). The N–W prediction grid (act_x1 and act_x2) is also included to generate posterior predictions across the response surface. Finally, c specifies the asymmetry parameter used in the LINEX loss function.

```
data_simu <- list(
  N  = nrow(df),
  N_test = nrow(grid),
  K = length(unique(df$year)),
  year = df$year,
  y  = df$profit,
  x1 = df$X1,
  x2 = df$X2,
  act_x1 = grid$N,
  act_x2 = grid$W,
  c = -0.005
)
```

After organizing the data, the model is fitted using four independent MCMC chains. Each chain uses 4,000 warmup iterations for sampler adaptation followed by 2,000 retained sampling iterations, resulting in 8,000 post-warmup posterior draws across all chains. A fixed random seed is used to ensure computational reproducibility. The `adapt_delta` parameter is set to 0.98 and `max_treedepth` to 12 to improve sampling in regions with more complex posterior geometry and reduce the occurrence of divergent transitions.

```
sim <- model$sample(
  data = data_simu,
  chains = 4,
  parallel_chains = 4,
  iter_warmup     = 2000,
  iter_sampling   = 1000,
  seed            = 1248,
  refresh         = 500,
  max_treedepth   = 12,
  adapt_delta     = 0.98,
  show_messages = F,
  show_exceptions = F
)
```

7 MCMC convergence diagnostics

The MCMC convergence diagnostics was evaluated through the posterior distributions of the model parameters. The posterior summaries of the numerical diagnostics reported by `sim$summary()` include the potential scale reduction factor $\widehat{R}$ and the bulk and tail effective sample sizes. Values of $\widehat{R}$ close to 1 indicate that the chains sampled the same stationary distribution.

```
mvars <- c("x_s", "eigen_val", "eigen_vec",
                    "b0", "b1", "b2", "b3", "b4", "b5",
                    "sigma", "alpha", "sigma_v", "Z_j")

resu <- sim$summary(mvars)

kable(resu, caption = "Summary", label = "summary", digits = 2) |>
  kable_styling(font_size = 10, full_width = FALSE)
```

The table 2 should be inspected to confirm that every monitored parameter and derived quantity satisfies the $\widehat{R}$ criterion and has sufficiently large effective sample sizes.

It is possible to verify that all parameters achieved satisfactory convergence through the trace plots (e.g. Figures 2, 3). Furthermore, β_1, β_3, and β_5, which are associated with the effects of X_1, present weak evidence of an effect since their posterior distributions are concentrated around zero (Figure 4. In contrast, β_0 exhibits a strong effect. It is worth emphasizing that all parameters in θ presented $\widehat{R} \approx 1$, indicating satisfactory convergence of the Markov chains.

Graphical diagnostics complement the numerical summaries. Trace plots display the sampled value against iteration for each chain. Well-mixed and stationary traces should overlap and fluctuate around a common region without trends, prolonged sticking, or systematic separation among chains. Posterior density plots summarize parameter location,

Table 2: Summary

variable	mean	median	sd	mad	q5	q95	rhat	ess_bulk	ess_tail
x_s[1]	0.88	0.82	0.57	0.47	0.13	1.86	1	4593.84	3221.09
x_s[2]	1.32	1.28	0.23	0.19	1.02	1.71	1	4916.65	2678.17
eigen_val[1]	-40.29	-40.16	3.89	3.89	-46.69	-34.04	1	3190.62	2689.72
eigen_val[2]	-8.11	-8.09	1.73	1.67	-10.99	-5.28	1	5808.09	3301.80
eigen_vec[1,1]	-0.28	-0.28	0.06	0.06	-0.38	-0.18	1	3871.30	2906.44
eigen_vec[2,1]	0.96	0.96	0.02	0.02	0.92	0.98	1	3871.31	2906.44
eigen_vec[1,2]	0.96	0.96	0.02	0.02	0.92	0.98	1	3871.31	2906.44
eigen_vec[2,2]	0.28	0.28	0.06	0.06	0.18	0.38	1	3871.30	2906.44
b0	254.70	254.73	11.57	11.45	235.68	274.11	1	1885.65	2510.11
b1	-4.57	-4.57	4.43	4.40	-11.88	2.53	1	4040.83	2753.41
b2	81.27	81.37	4.99	4.98	72.85	89.12	1	3475.03	3083.56
b3	-10.79	-10.79	2.05	2.02	-14.16	-7.47	1	4172.00	3042.59
b4	-37.61	-37.55	3.53	3.54	-43.53	-31.89	1	3655.46	2893.63
b5	17.30	17.19	4.46	4.32	10.04	24.88	1	3246.44	2708.80
sigma	107.20	107.19	3.04	3.03	102.18	112.10	1	3700.11	3010.83
alpha	0.97	0.98	0.14	0.13	0.74	1.20	1	3046.93	2662.97
sigma_v	38.71	38.35	4.94	4.84	31.14	47.41	1	2577.89	2969.76
Z_j[1]	1.16	1.15	0.43	0.42	0.49	1.88	1	2249.51	2568.59
Z_j[2]	2.69	2.68	0.46	0.45	1.95	3.48	1	2218.84	2712.22
Z_j[3]	-1.69	-1.68	0.48	0.48	-2.51	-0.95	1	2155.03	2398.44
Z_j[4]	0.38	0.38	0.37	0.37	-0.22	1.00	1	1953.48	2716.60
Z_j[5]	2.21	2.20	0.41	0.41	1.56	2.90	1	2013.22	2542.05
Z_j[6]	-1.60	-1.57	0.43	0.42	-2.34	-0.92	1	2039.70	2495.15

spread, skewness, and possible multimodality. Density plots alone do not establish convergence and must be interpreted together with trace plots, $\widehat{R}$, effective sample sizes, and sampler diagnostics.

```
draws <- sim$draws(mvars)
```

```
mcmc_trace(draws,  pars = c("b0", "b1", "b2", "b3", "b4", "b5")) +
  scale_x_continuous(n.breaks = 3)
```

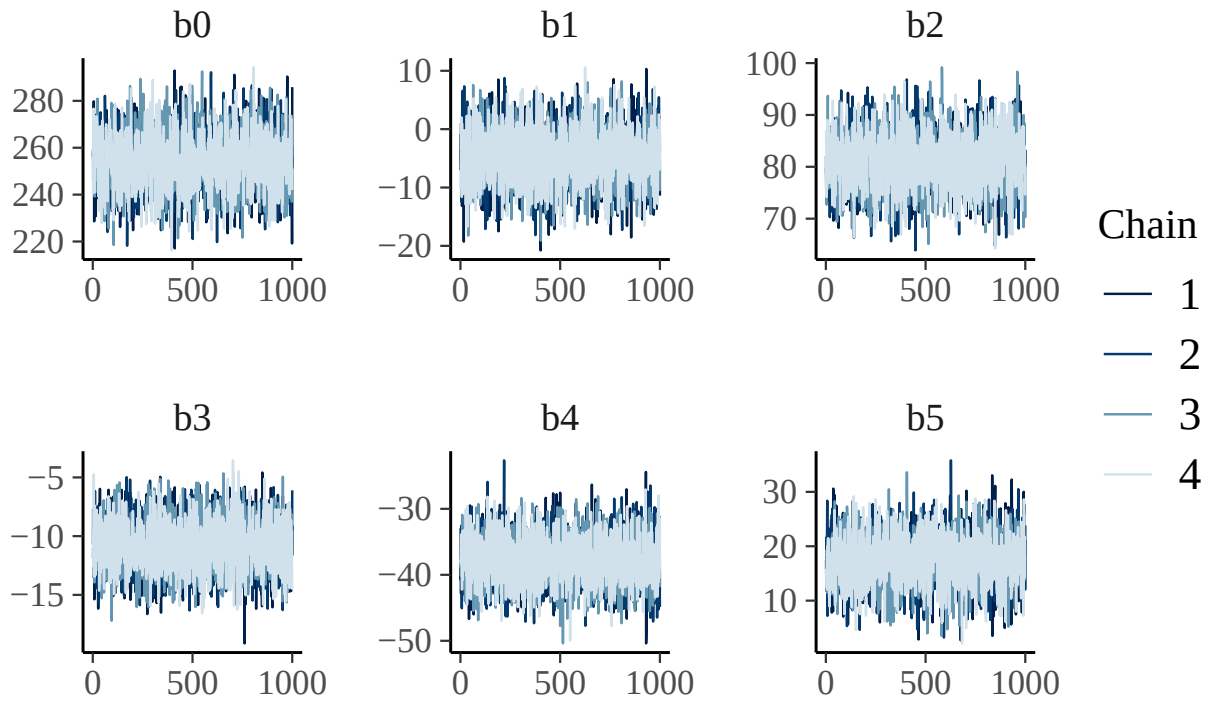


Figure 2: Trace plots for the linear predictor

```
mcmc_trace(draws, pars = c("x_s[1]", "x_s[2]", "eigen_val[1]", "eigen_val[2]",
                           "eigen_vec[1,1]", "eigen_vec[2,1]",
                           "eigen_vec[1,2]", "eigen_vec[2,2]")) +
  scale_x_continuous(n.breaks = 3) +
  scale_y_continuous(n.breaks = 3)
```

```
mcmc_intervals(draws,
  pars = c("alpha", "b0", "b1", "b2", "b3","b4",
           "b5", "sigma", "sigma_v", "x_s[1]", "x_s[2]"))
```

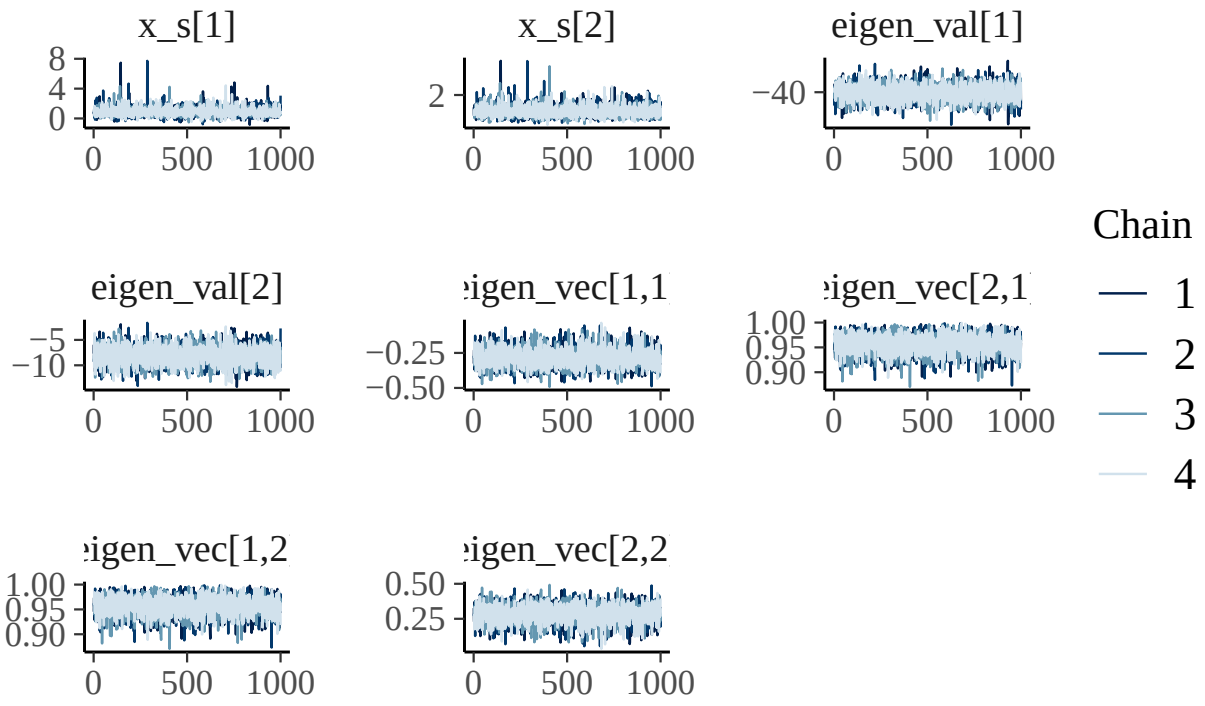


Figure 3: Trace plots for the canonical parameters

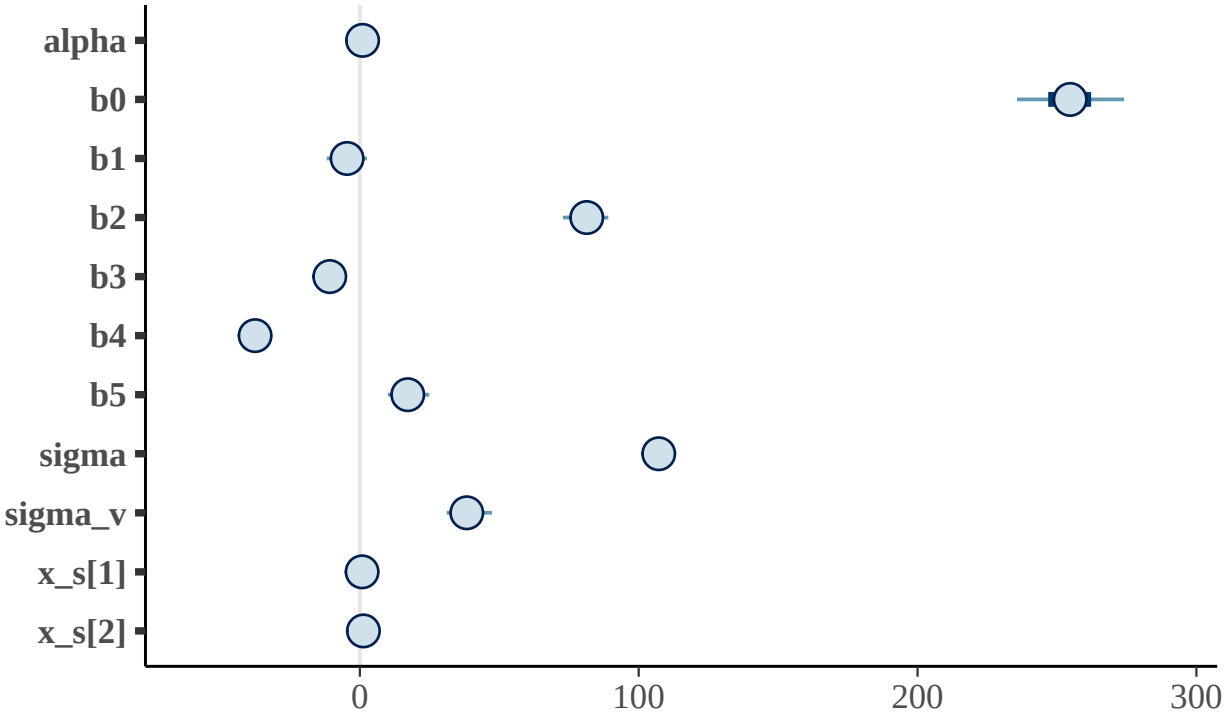


Figure 4: MCMC Intervals

8 Posterior predictive checking

Posterior predictive checking evaluates whether data simulated from the fitted model reproduce important characteristics of the observed profit distribution. For every retained posterior draw, `y_rep` contains a replicated dataset generated from the skew-normal distribution. Figure 5 overlays the observed density with densities from 100 posterior predictive replicates.

Given the posterior distribution of profit, we can further assess the model fit through the posterior predictive distribution shown in Figure @ref(fig:post_profit). The figure presents 100 posterior predictive draws together with the density of the observed profit values. The close agreement between the posterior predictive samples and the observed distribution suggests that the proposed model adequately captures the variability of the profit response.

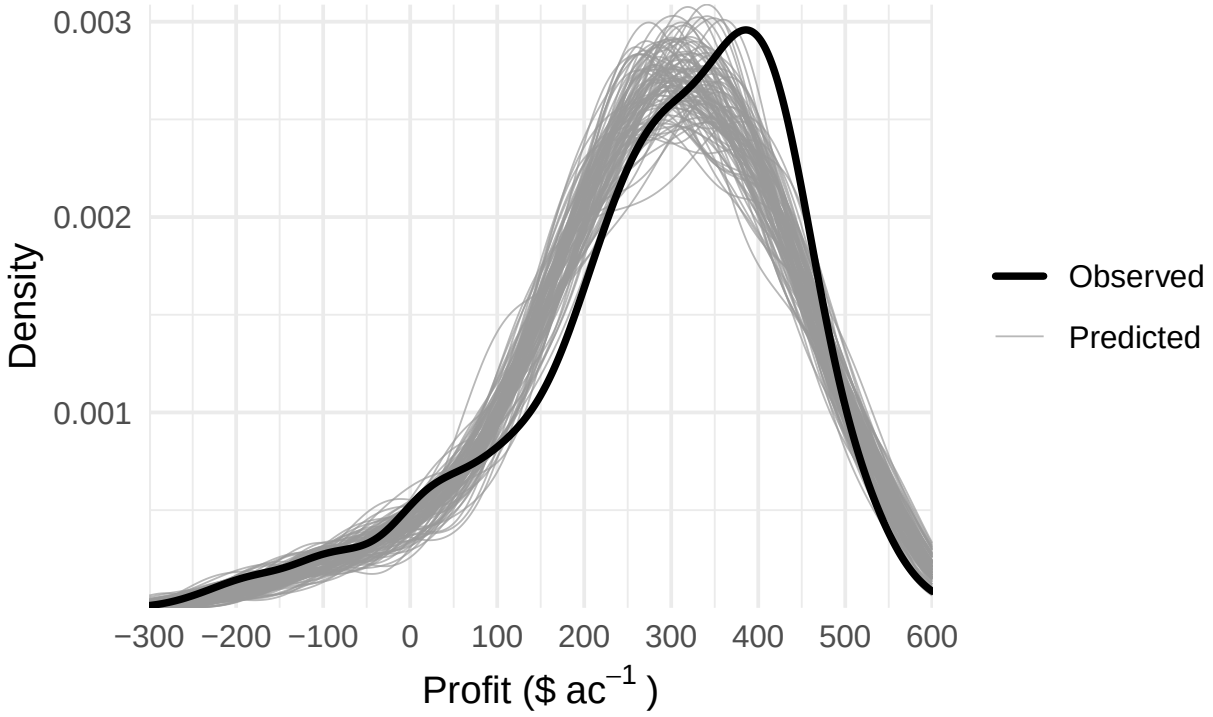
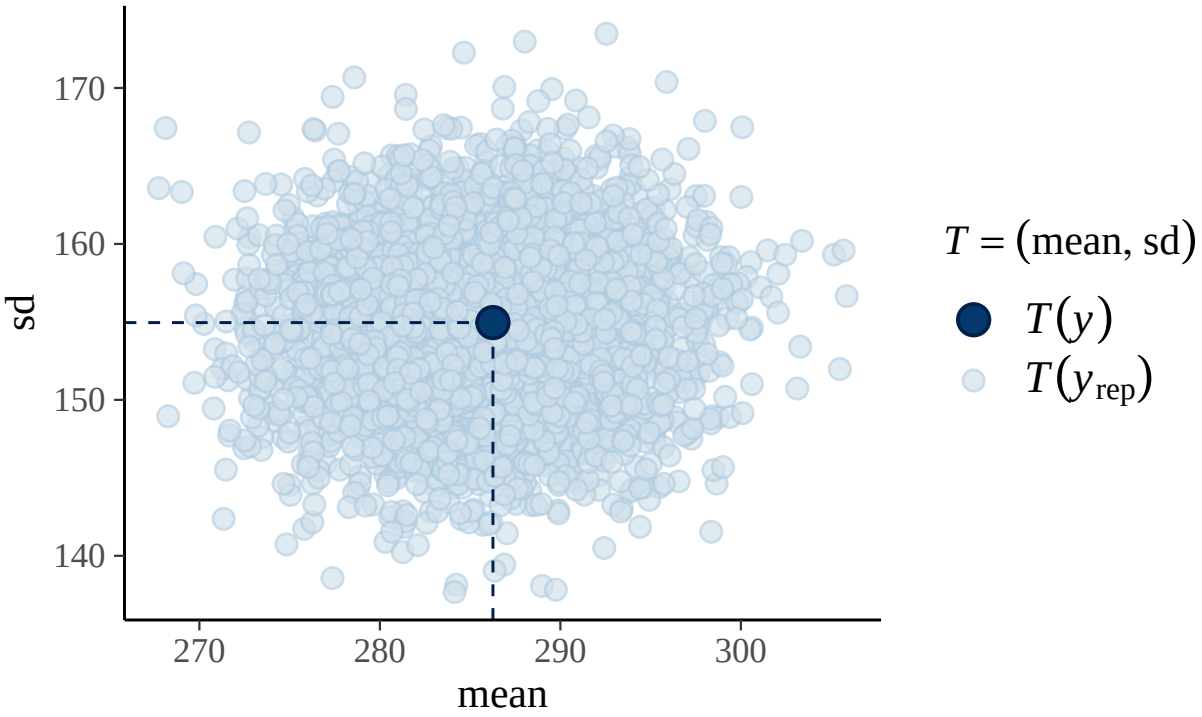


Figure 5: Posterior predictive density check.



References

Azzalini, Adelchi. 2013. *The Skew-Normal and Related Families*. Vol. 3. Cambridge University Press.

Brooks, Stephen. 1998. "Markov Chain Monte Carlo Method and Its Application." *Journal of the Royal Statistical Society: Series D (the Statistician)* 47 (1): 69–100.

Cressie, Noel. 2023. "Decisions, Decisions, Decisions in an Uncertain Environment." *Environmetrics* 34 (1): e2767.

Gelman, Andrew, John B Carlin, Hal S Stern, David B Dunson, Aki Vehtari, and Donald B Rubin. 2013. *Bayesian Data Analysis*. Vol. 2. Chapman & Hall/CRC.

Little, Roderick J. 2006. "Calibrated Bayes: A Bayes/Frequentist Roadmap." *The American Statistician* 60 (3): 213–23.

R Core Team. 2026. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. https://www.R-project.org/.

Stan Development Team. 2026. *Stan Reference Manual*. https://mc-stan.org.